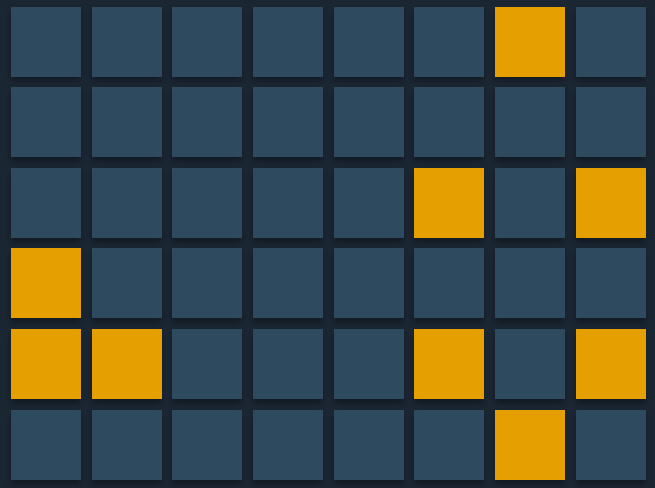


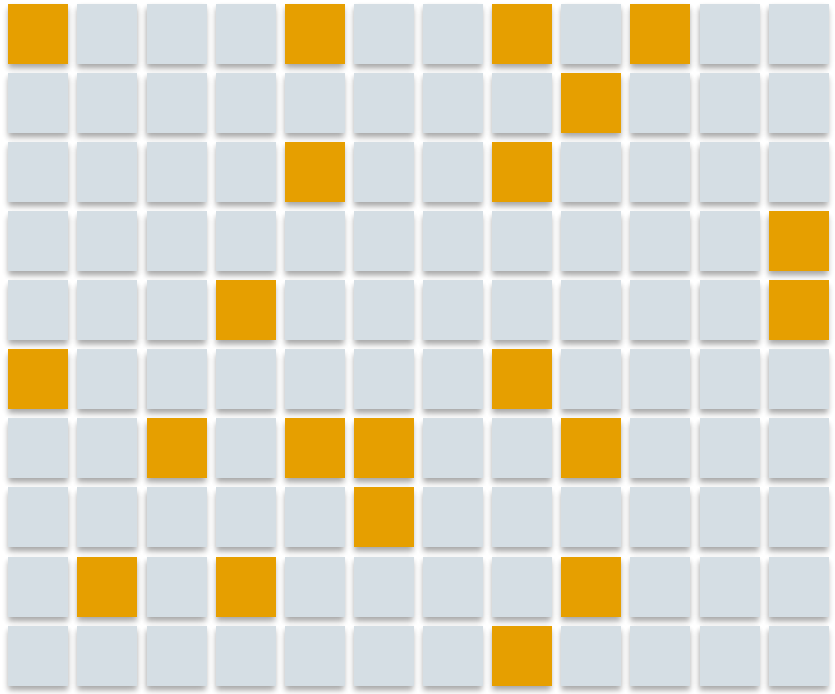
Do Python cellwise-robust PCA tools agree with R?

Independent validation against R reference code, p ≫ n and heavy contamination

Progress presentation · HDDA course, Industrial track, Direction 5

Team 4: Lev Lesnoi · Dinara Bairamgazieva · Semen Vashurin

20% bad cells touch almost every row; Python tools lack an R check



Orange = contaminated cell (20%)

Rows fully clean: 1.2% at p = 20, ≈ 0% at p = 200

Cellwise outliers

- single corrupted entries; rowwise-robust PCA discards whole rows and runs out of clean ones

New Python tools

- robustcov CellPCA, RobPy DDC → ROBPCA, PCP scripts

Gap

- none is validated against the R reference (cellWise::MacroPCA, authors' cellPCA code), especially for p ≫ n

Do Python implementations agree with R, and where do they fail?

Research question

Do Python cellwise-robust PCA implementations agree with the R reference?

Where (n, p, contamination) do they fail?

Pre-fixed criteria (proposal)

- R reproduces the paper: error ≥ 3× below PCA at γ = 6, p = 20 and 200
- “agrees”: median angle to R subspace ≤ max(1°, 95th pct of R-vs-R angles)

HYPOTHESIS

robustcov CellPCA behaves differently from the published cellPCA behaviour at p ≈ 20

- Basis: one earlier run of our approximation — at γ = 4, p = 20: CellPCA 32.5° vs PCA 26.0° (same earlier run, robustcov 0.2.0)
- A lead, not a verdict: R reference now run (slide 6); robustcov itself still needs a re-run

We start from the cellPCA paper's synthetic setting, then real data

Synthetic: cellPCA paper setting

our approximation, not the authors' generator

- $n = 100$, $p \in \{20, 200\}$, $q = 2$
- Σ: two dominant eigenvalues (9.57, 6.70 | 104.88, 73.42), rest ≈ 0.1, random eigenvectors
- 20% of cells shifted by $\gamma \cdot \sigma_j$, $\gamma \in \{0, 2, 4, 6\}$
- 25 runs per setting, seeds 1000 + s

TopGear (RobPy)

297 cars × 13 numeric, 110 missing cells

dims checked today

Glass spectra (RobPy)

180 × 750, p > n

dims checked today

Golub leukemia (OpenML 45090)

72 × 7129, outliers injected

link not yet verified

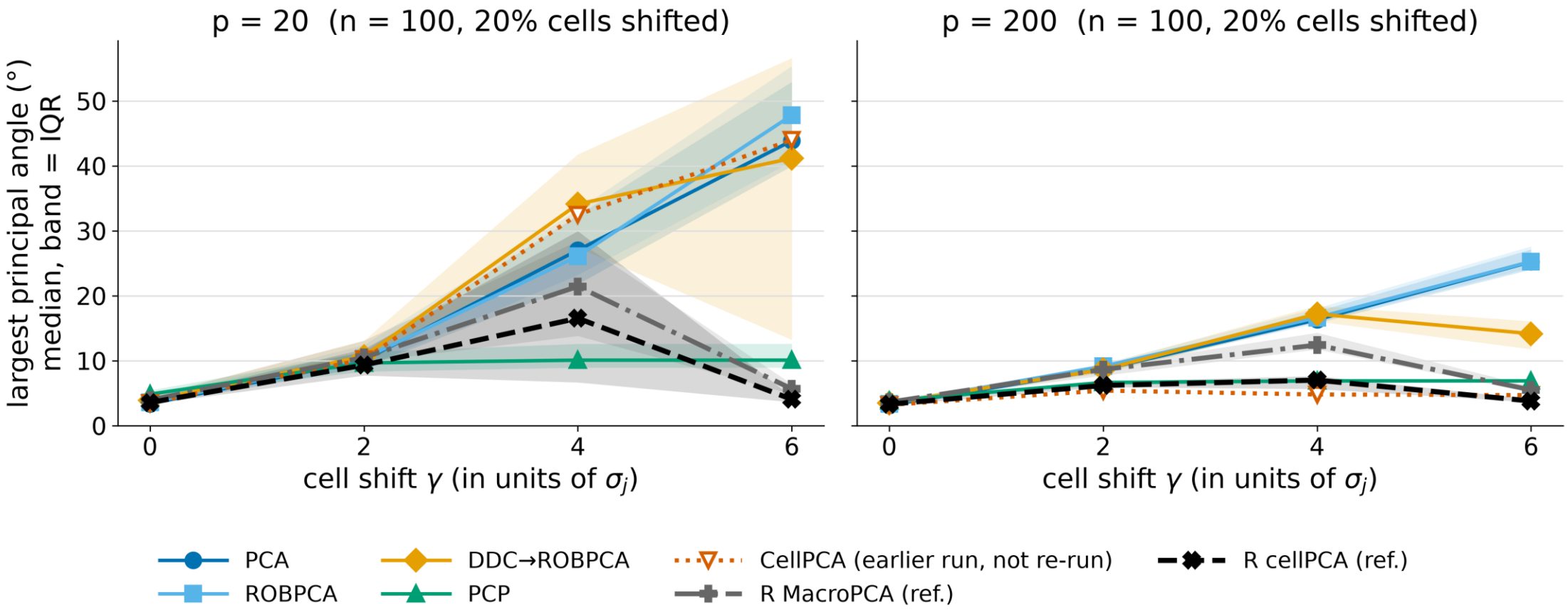
R is the reference, not a competitor; the metric is one angle

Method	Role	This week
R cellPCA (authors' code) · R cellWise::MacroPCA	Reference (conformance)	run: 25 / 25 seeds
robustcov CellPCA	System under test	not installable from PyPI as of 4 Nov 2026; earlier-run medians
RobPy DDC → ROBPCA (manual two-step)	System under test	run: 25 seeds
PCP (dfm/pcp)	System under test (low-rank + sparse)	run: 25 seeds
RobPy ROBPCA, spherical PCA; sklearn PCA	Baselines / lower bound	run: 25 seeds

Metric: largest principal angle between estimated and true 2-D subspace (degrees)

0° = same subspace · 90° = some true direction completely missed · median over runs, band = IQR

R reproduces the paper; the earlier CellPCA run did not at p = 20



Reading: at γ = 6, R cellPCA 4.0° / 3.8° vs PCA 43.9° / 25.2° (p = 20 / 200) → paper behaviour reproduced (25 runs per point).

Lead: earlier CellPCA run (not re-run) gave 44.0° at p = 20, close to PCA — unlike R cellPCA.

How the angle is computed: three lines of linear algebra

1

Orthonormalise

$Q = \mathrm{qr}(V)$

V: p × 2 loadings from any method

2

Cosines of angles

$\sigma =$ singular values of $Q^T U$

U: true orthonormal basis

3

Largest angle

$\theta = \arccos(\min \sigma)$

in degrees; 0° = perfect

- Invariant to sign flips, rotations inside the subspace and loading scale → comparable across packages and languages
- Same function scores Python and R outputs; R loadings come back via CSV (Rscript, no rpy2)

The R reference works; robustcov could not be re-run this week

What the results say

- R gate passed: PCA/R-cellPCA error ratio at γ = 6 is 10.8× (p = 20) and 6.7× (p = 200)
- TopGear vignette reproduced: 126 flagged cells, cutoffOD 1.737869
- CellPCA lead (not re-run): clear at γ = 6, p = 20 (44.0° vs R 4.0°); at γ = 4 R itself varies (IQR 6.7–30.0°)
- DDC → ROBPCA (RobPy defaults) no better than PCA up to γ = 4 (p = 200: 17.2° vs 16.3°); adapter settings to be checked
- PCP flat at ≈ 10.1° / 6.9°: different model (low-rank + sparse)

Challenges

- robustcov: not installable from PyPI as of 4 Nov 2026, status being clarified
- Our setting approximates the paper (positive shifts, 3rd eigenvalue 0.11)
- R cellPCA cost: ~5.5 s/run at p = 200 (MacroPCA 0.4 s) → R only up to p ≤ 1000
- Python DDC: ~39.6 s/run at p = 200, no fastDDC

timings: wall time, several runs in parallel on one laptop

Six weeks to 18 Dec: conformance tests, suite, failure map

Wk 1–2	R reference on full grid; R-vs-R spread; conformance tests; re-run robustcov once available
Wk 3	Adapters as pip-installable suite; tests ≥ 80% coverage
Wk 4	Reduced grid (R only up to $p \leq 1000$); failure map
Wk 5	Real data: TopGear, glass, Golub; reproducible reports to maintainers
Wk 6	Documentation, worked examples, final report (18 Dec)

Pre-fixed tolerance: a Python method agrees with R if its median angle to the R subspace ≤ max(1°, 95th percentile of R-vs-R angles over 25 seeds)